

UNIT 3:

Transaction Processing & Concurrency Control

Non-Serializable Schedules

- A non-serial schedule which is not serializable is called as a non-serializable schedule.
- A non-serializable schedule is not guaranteed to produce the the same effect as produced by some serial schedule on any consistent database.

- **Characteristics-**

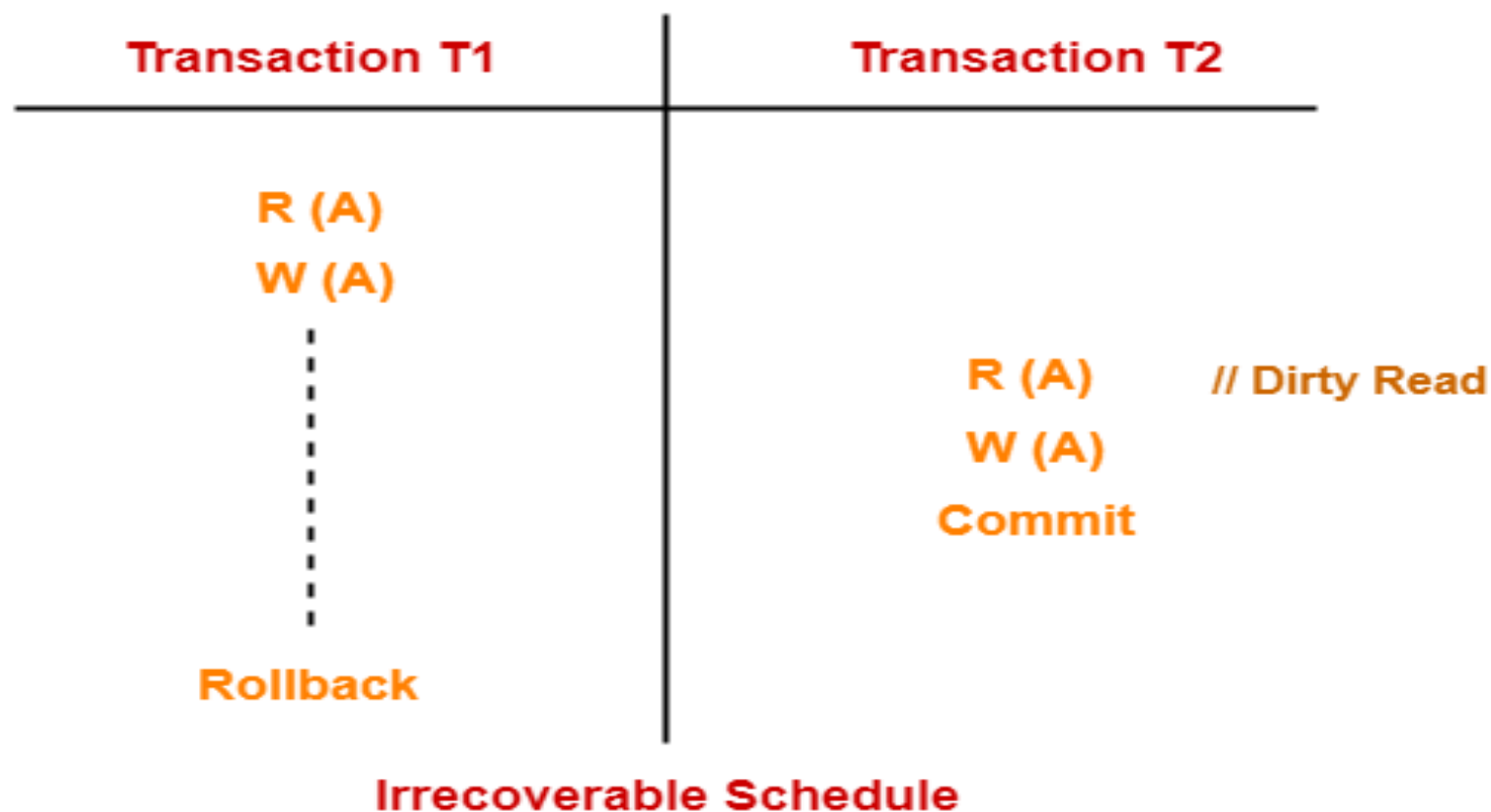
Non-serializable schedules-

- may or may not be consistent
- may or may not be recoverable

Irrecoverable Schedules-

If in a schedule,

- A transaction performs a dirty read operation from an uncommitted transaction
- And commits before the transaction from which it has read the value then such a schedule is known as an **Irrecoverable Schedule**.



Here,

- T2 performs a dirty read operation.
- T2 commits before T1.
- T1 fails later and roll backs.
- The value that T2 read now stands to be incorrect.
- T2 can not recover since it has already committed.

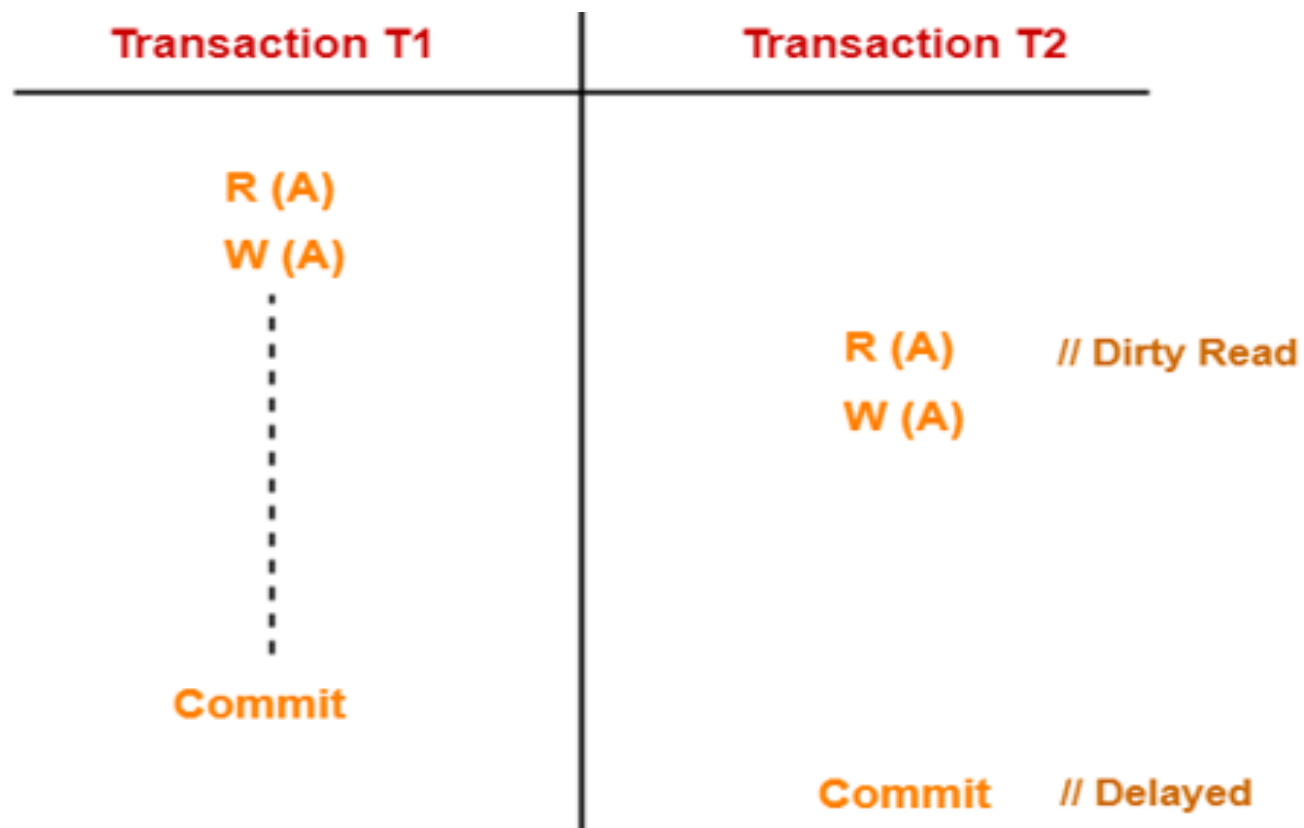
Recoverable Schedules-

If in a schedule,

- A transaction performs a dirty read operation from an uncommitted transaction
- And its commit operation is delayed till the uncommitted transaction either commits or roll backs then such a schedule is known as a **Recoverable Schedule**.

Here,

- The commit operation of the transaction that performs the dirty read is delayed.
- This ensures that it still has a chance to recover if the uncommitted transaction fails later.



Recoverable Schedule

Here,

- T2 performs a dirty read operation.
 - The commit operation of T2 is delayed till T1 commits or roll backs.
 - T1 commits later.
-
- T2 is now allowed to commit.
 - In case, T1 would have failed, T2 has a chance to recover by rolling back.

Checking Whether a Schedule is Recoverable or Irrecoverable-

- METHOD 1
- Check whether the given schedule is conflict serializable or not.
- If the given schedule is conflict serializable, then it is surely recoverable. Stop and report your answer.
- If the given schedule is not conflict serializable, then it may or may not be recoverable. Go and check using other methods.

-

Thumb Rules

- All conflict serializable schedules are recoverable.
- All recoverable schedules may or may not be conflict serializable.

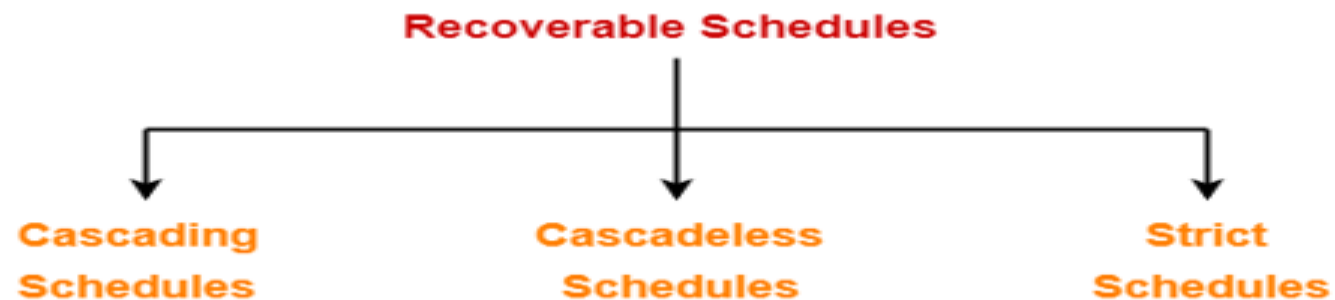
- **Method-02:**
- Check if there exists any dirty read operation.
- (Reading from an uncommitted transaction is called as a dirty read)
- If there does not exist any dirty read operation, then the schedule is surely recoverable. Stop and report your answer.
- If there exists any dirty read operation, then the schedule may or may not be recoverable.
- If there exists a dirty read operation, then follow the following cases-
- **Case-01:**
- If the commit operation of the transaction performing the dirty read occurs before the commit or abort operation of the transaction which updated the value, then the schedule is irrecoverable.
- **Case-02:**
- If the commit operation of the transaction performing the dirty read is delayed till the commit or abort operation of the transaction which updated the value, then the schedule is recoverable.

- **Recoverable Schedules-**

- If in a schedule,
- A transaction performs a dirty read operation from an uncommitted transaction
- And its commit operation is delayed till the uncommitted transaction either commits or roll backs then such a schedule is called as a **Recoverable Schedule**.

- **Types of Recoverable Schedules-**

- A recoverable schedule may be any one of these kinds-



- **Cascading Schedule-**

- If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.
- It simply leads to the wastage of CPU time.

T1	T2	T3	T4
R (A)			
W (A)			
	R (A)		
	W (A)		
		R (A)	
		W (A)	
			R (A)
			W (A)
Failure			

Cascading Recoverable Schedule

Here,

- Transaction T2 depends on transaction T1.
- Transaction T3 depends on transaction T2.
- Transaction T4 depends on transaction T3.

In this schedule,

- The failure of transaction T1 causes the transaction T2 to rollback.
- The rollback of transaction T2 causes the transaction T3 to rollback.
- The rollback of transaction T3 causes the transaction T4 to rollback.

Such a rollback is called as a **Cascading Rollback**.

NOTE-

If the transactions T2, T3 and T4 would have committed before the failure of transaction T1, then the schedule would have been irrecoverable.

- **Cascadeless Schedule-**

- If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Cascadeless Schedule**.
- In other words,
- Cascadeless schedule allows only committed read operations.
- Therefore, it avoids cascading roll back and thus saves CPU time.

T1	T2	T3
R (A)		
W (A)		
Commit		
	R (A)	
	W (A)	
	Commit	
		R (A)
		W (A)
		Commit

Cascadeless Schedule

NOTE-

- Cascadeless schedule allows only committed read operations.
- However, it allows uncommitted write operations.

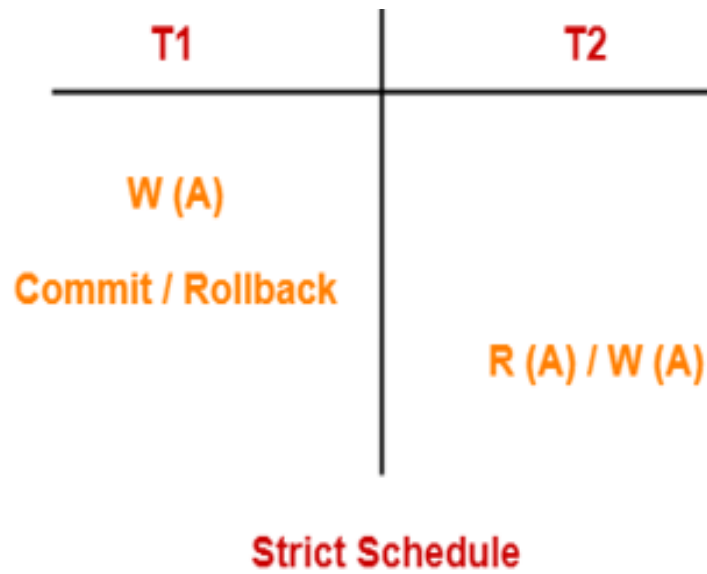
Example-

T1	T2
R (A)	
W (A)	
	W (A) // Uncommitted Write
Commit	

Cascadeless Schedule

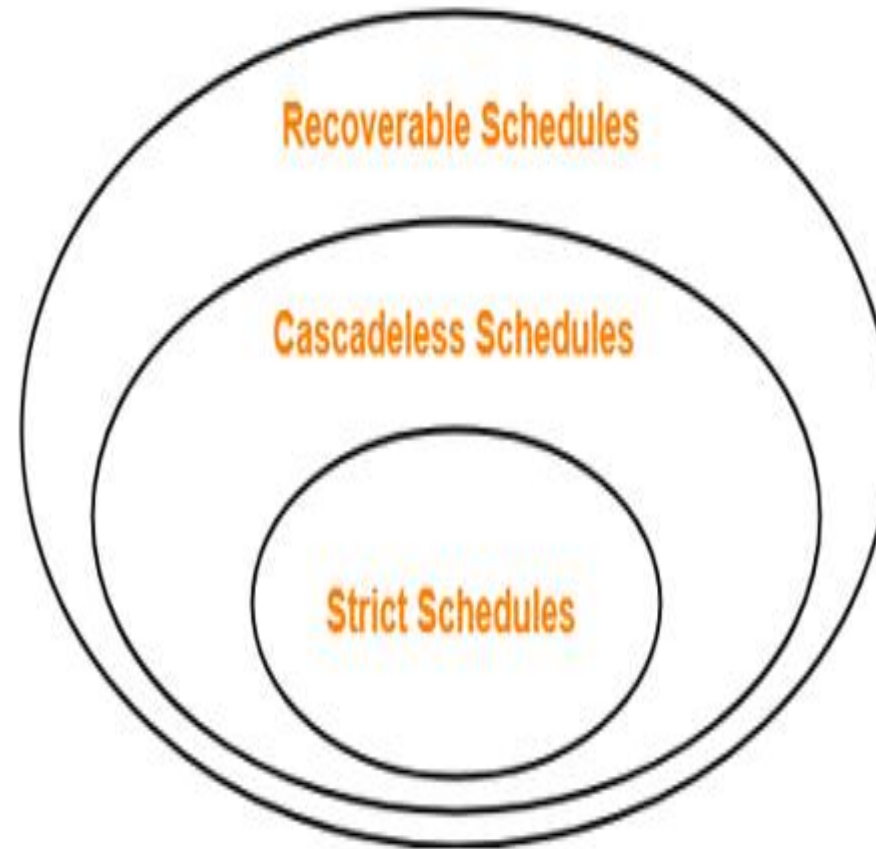
- **Strict Schedule-**

- If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Strict Schedule**.
- In other words,
- Strict schedule allows only committed read and write operations.
- Clearly, strict schedule implements more restrictions than cascadeless schedule.



Remember-

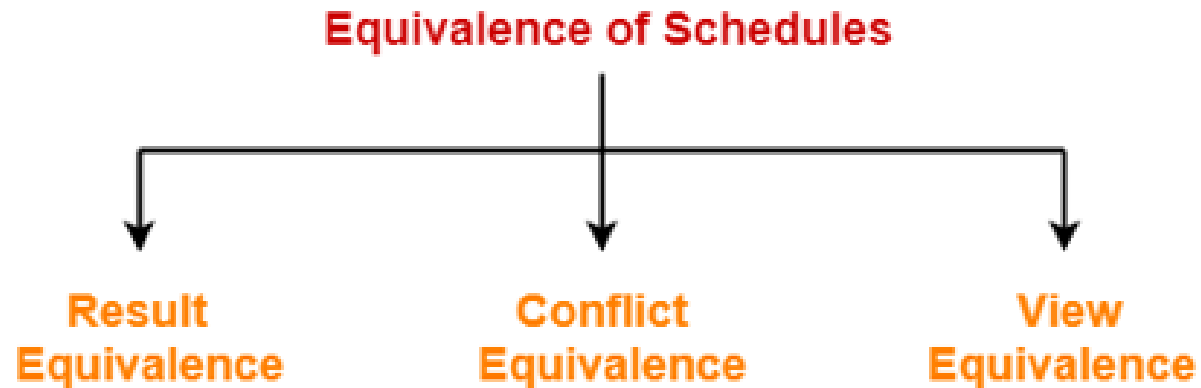
- Strict schedules are more strict than cascadeless schedules.
- All strict schedules are cascadeless schedules.
- All cascadeless schedules are not strict schedules.



- **Equivalence of Schedules-**

- In DBMS, schedules may have the following three different kinds of equivalence relations among them-

-



1. Result Equivalent Schedules-

- If any two schedules generate the same result after their execution, then they are called as result equivalent schedules.
- This equivalence relation is considered of least significance.
- This is because some schedules might produce same results for some set of values and different results for some other set of values.

2. Conflict Equivalent Schedules-

- If any two schedules satisfy the following two conditions, then they are called as conflict equivalent schedules-
- The set of transactions present in both the schedules is same.
- The order of pairs of conflicting operations of both the schedules is same.

3. View Equivalent Schedules-

We have already discussed about [View Equivalent Schedules](#)

- Are the following three schedules result equivalent?

Schedule S1		Schedule S2		Schedule S3	
T1	T2	T1	T2	T1	T2
R (X)		R (X)			R (X)
X = X + 5		X = X + 5			X = X x 3
W (X)		W (X)			W (X)
R (Y)			R (X)	R (X)	
Y = Y + 5			X = X x 3	X = X + 5	
W (Y)			W (X)	W (X)	
	R (X)	R (Y)		R (Y)	
	X = X x 3	Y = Y + 5		Y = Y + 5	
	W (X)	W (Y)		W (Y)	

- To check whether the given schedules are result equivalent or not,
- We will consider some arbitrary values of X and Y.
- Then, we will compare the results produced by each schedule.
- Those schedules which produce the same results will be result equivalent.
- Let $X = 2$ and $Y = 5$.
- On substituting these values, the results produced by each schedule are-
- **Results by Schedule S1-** $X = 21$ and $Y = 10$
- **Results by Schedule S2-** $X = 21$ and $Y = 10$
- **Results by Schedule S3-** $X = 11$ and $Y = 10$
- Clearly, the results produced by schedules S1 and S2 are same.
- Thus, we conclude that S1 and S2 are result equivalent schedules.

- Are the following two schedules conflict equivalent?

T1	T2
R (A)	
W (A)	
	R (A)
	W (A)
R (B)	
W (B)	

Schedule S1

T1	T2
R (A)	
W (A)	
R (B)	
W (B)	
	R (A)
	W (A)

Schedule S2

- To check whether the given schedules are conflict equivalent or not,
- We will write their order of pairs of conflicting operations.
- Then, we will compare the order of both the schedules.
- If both the schedules are found to have the same order, then they will be conflict equivalent.
- **For schedule S1-**
- The required order is-
- $R_1(A)$, $W_2(A)$
- $W_1(A)$, $R_2(A)$
- $W_1(A)$, $W_2(A)$
- **For schedule S2-**
- The required order is-
- $R_1(A)$, $W_2(A)$
- $W_1(A)$, $R_2(A)$
- $W_1(A)$, $W_2(A)$
- Clearly, both the given schedules have the same order.
- Thus, we conclude that S1 and S2 are conflict equivalent schedules.

Is the following schedule S is a recoverable schedule?

Sl.No.	T1	T2
1		W(A)
2	W(B)	
3	W(A)	
4		R(B)
5	Commit	
6		Commit

Is the following schedule is cascadeless or not?

Instruction	T ₁	T ₂
1	$R(x)$	$R(x)$
2	$W(y)$	
3		$W(y)$
4	$Commit$	$Commit$
5		
6		

This schedule is cascadeless because there is no dirty read.

Is the following schedule is cascadeless or not?

Instruction	T ₁	T ₂	T ₃
1	R(x)		
2	R(y)		
3	W(x)	R(x)	
4		W(x)	R(x)
5			
6			

This schedule is said to be a partial schedule as it does not consist of either commit or abort statements. In this schedule, if T₁ fails and aborted then T₂ which is dependent on T₁ has to be rolled back and T₃ which is dependent on T₂ has also to be rolled back. This type of rollback is called as cascading rollback. Hence, this schedule is not a cascadeless schedule.

- Which concurrency control protocols ensure conflict serializability and freedom from the deadlock?

- A) 2-phase locking
- B) Time-stamp Ordering
- C) Both 2-phase locking and Time-stamp ordering
- D) None of the above

ANSWER:- B

Explanation :2PL, i.e., [2 Phase Locking](#), is a concurrency control method that guarantees serializability. This protocol uses locks applied to data by a transaction, which blocks other transactions from accessing the same data during the transaction's life. Mutual blocking of 2 or more transactions may lead to deadlock in 2PL.

- Which among the following statements is/are incorrect?
 - 1) A schedule following a strict two-phase locking protocol is conflict serializable and recoverable.
 - 2) Checkpoint in schedules inserted to ensure recoverability.

- A) Only 1
- B) Only 2
- C) Both 1 and 2
- D) None of the above

ANSWER:- B

Explanation : A basic two-phase [locking protocol](#) ensures only conflict serializability. In contrast, a strict two-phase locking protocol ensures conflict serializability and recoverability. So statement (1) is correct. i.e., checkpoints are inserted to minimize the task of undo-redo in recoverability. And hence statement (2) is not correct. So Option (B) is correct.